

Password Strength Checker

1. Project Overview

The **Password Strength Checker** is a Python-based application that evaluates the strength of a user's password based on predefined security requirements.

The program checks whether a password:

- Contains at least **8 characters**
- Contains at least **one number**
- Contains at least **one uppercase letter**
- Contains at least **one lowercase letter**
- Contains at least **one special character**

Based on these conditions, the application classifies the password as **Weak**, **Medium**, or **Strong**.

2. Objectives

The main objectives of this project are:

1. To create a simple password security-checking application.
 2. To help users understand whether their password meets basic security requirements.
 3. To demonstrate the use of Python functions, loops, conditional statements, and regular expressions.
 4. To provide meaningful feedback when a password does not meet a particular requirement.
 5. To encourage users to create stronger and more secure passwords.
-

3. Technologies Used

Technology	Purpose
Python	Main programming language
re module	Checking for special characters
input()	Accepting passwords from the user
Conditional statements	Evaluating password requirements
Functions	Organizing the program into reusable components
while loop	Allowing repeated password checks

4. System Requirements

Hardware Requirements

- Computer or laptop
- Keyboard
- Minimum 2 GB RAM recommended

Software Requirements

- Python 3.x
- Any Python-compatible IDE or code editor, such as:
 - VS Code
 - PyCharm
 - IDLE
 - Jupyter Notebook

No external Python packages are required because the project uses Python's built-in re module.

5. Password Strength Criteria

The application evaluates passwords using the following rules:

Requirement	Description
Minimum length	Password must contain at least 8 characters
Number	Password must contain at least one digit (0–9)
Uppercase	Password must contain at least one uppercase letter (A–Z)
Lowercase	Password must contain at least one lowercase letter (a–z)
Special character	Password must contain at least one special character

Strength Classification

Weak

A password is considered weak if it fails any of the basic requirements, such as minimum length, number, uppercase letter, or lowercase letter.

Medium

A password that satisfies the basic requirements but does not contain a special character is classified as medium.

Strong

A password that satisfies all five requirements is classified as strong.

7. Program Components

7.1 Importing the Regular Expression Module

```
import re
```

The `re` module is used to search the password for special characters.

7.2 `check_password_strength()` Function

```
def check_password_strength(password):
```

This function receives a password as input and evaluates it against all predefined conditions.

Length Check

```
if len(password) < 8:
```

```
    return "Weak: Password must be at least 8 characters long."
```

This ensures that the password contains at least eight characters.

Number Check

```
if not any(char.isdigit() for char in password):
```

```
    return "Weak: Password must include at least one number."
```

The `isdigit()` method checks whether at least one character is a digit.

Uppercase Check

```
if not any(char.isupper() for char in password):
```

```
    return "Weak: Password must include at least one uppercase letter."
```

The `isupper()` method checks for an uppercase letter.

Lowercase Check

```
if not any(char.islower() for char in password):
```

```
    return "Weak: Password must include at least one lowercase letter."
```

The `islower()` method checks for a lowercase letter.

Special Character Check

```
if not re.search(r'[!@#$%^&*(),.?":{}|<>]', password):
```

```
    return "Medium: Add special characters to make your password stronger."
```

The regular expression searches for at least one special character.

Strong Password

```
return "Strong: Your password is secure!"
```

If all requirements are satisfied, the function returns a strong-password message.

8. Main Function

The `password_checker()` function handles interaction with the user.

```
def password_checker():
```

It displays a welcome message and continuously asks the user to enter a password.

A while True loop is used so that the user can check multiple passwords without restarting the program.

The user can type:

exit

to terminate the program.

9. Sample Input and Output

Example 1: Short Password

Input:

hello

Output:

Weak: Password must be at least 8 characters long.

Example 2: Missing Special Character

Input:

Hello123

Output:

Medium: Add special characters to make your password stronger.

Example 3: Strong Password

Input:

Hello123!

Output:

Strong: Your password is secure!

Example 4: Exit Program

Input:

exit

Output:

Thank you for using the Password Strength Checker! Goodbye!

10. Advantages

- Simple and easy to use.
 - Beginner-friendly Python project.
 - Provides immediate feedback to the user.
 - Uses reusable functions.
 - Does not require external libraries.
 - Demonstrates practical use of regular expressions.
 - Helps users understand basic password security requirements.
-

11. Limitations

The current version has some limitations:

1. It only checks basic password requirements.
 2. It does not check whether the password appears in a database of commonly used or compromised passwords.
 3. It does not measure password entropy.
 4. It does not check for repeated or predictable patterns.
 5. The password is entered using the normal input() function, so it may be visible on the screen.
 6. The special-character list is limited to the characters defined in the regular expression.
-

12. Future Enhancements

The project can be improved by adding:

- Hidden password input using Python's `getpass` module.
 - A password strength score, such as 0–100.
 - A graphical user interface (GUI).
 - Detection of common passwords.
 - Detection of repeated characters and predictable patterns.
 - Password suggestions or secure password generation.
 - A password history or security report.
 - More comprehensive special-character support.
 - Entropy-based password strength estimation.
-

13. Security Considerations

This project is intended for **basic password-strength education and checking**. It should not be considered a complete password-security solution.

For real-world applications:

- Passwords should never be stored in plain text.
 - Passwords should not be printed or logged.
 - Passwords should be securely hashed before storage.
 - Multi-factor authentication should be considered.
 - Passwords should be checked against known compromised-password databases where appropriate.
-

14. Conclusion

The **Password Strength Checker** is a simple Python project designed to evaluate passwords against common security requirements.

The project demonstrates important Python programming concepts, including:

- Functions
- Conditional statements
- Loops
- String methods
- The `any()` function
- Regular expressions
- User input and output

Although the application is simple, it provides a useful introduction to password security and demonstrates how programming can be used to solve practical security-related problems.

15. Author

Project Name: Password Strength Checker

Programming Language: Python

Project Type: Beginner Python / Cybersecurity Project